2026-2027 学年第一学期

《高级语言程序设计(C++语言)》

实验报告

实验名称：函数的定义、调用与调试	
实验日期：2026 年 10 月 30 日	实验地点：于刚宋晓楼
学　　号：2024301051244	姓　　名：卫金男
学　　院：	专　　业：人工智能专业
指导老师：	

一、实验目的

掌握函数的声明、定义和调用，比较值参数与引用参数，练习函数重载、默认参数和内联函数。

完成独立控制台程序，使用 Visual Studio 的断点、逐语句和逐过程功能观察形参、局部变量与递归调用。

练习使用标准库函数，例如用 std::isfinite 检查浮点数是否为有限值。

二、实验环境

实验在本地 Windows 上完成，使用 Visual Studio Community 2026，编译器为 MSVC 19.51。

每题建立独立的 C++ 控制台项目，采用 Debug x64 配置和 C++17 标准。

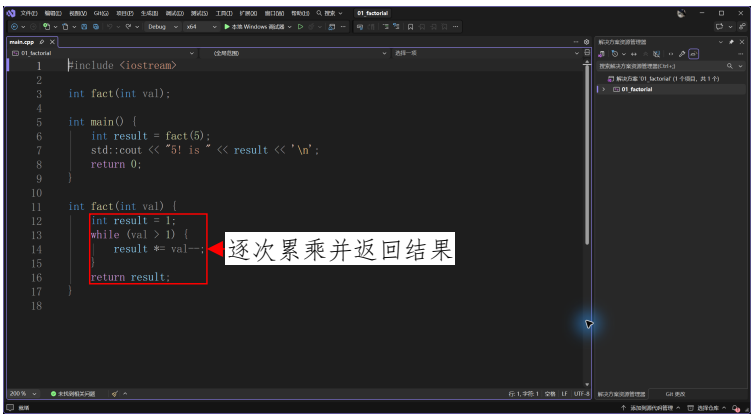
运行结果通过标准输出重定向保存到 output.txt，再在 Visual Studio 中打开文件截图。

三、实验过程及结果

题目 1　课堂练习：阶乘函数

将阶乘计算写为独立函数，求 5!；比较单步进入与单步跳过，观察形参的变化和作用域，并分别检查四种调用。

main 调用 fact(5)。fact 将累积值设为 1，每轮先乘当前 val，再递减 val，最后返回乘积。




```cpp
#include <iostream>

int fact(int val);

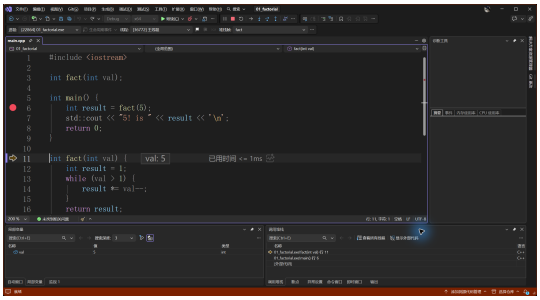
int main() {
    int result = fact(5);
    std::cout << "5! is " << result << '\n';
    return 0;
}

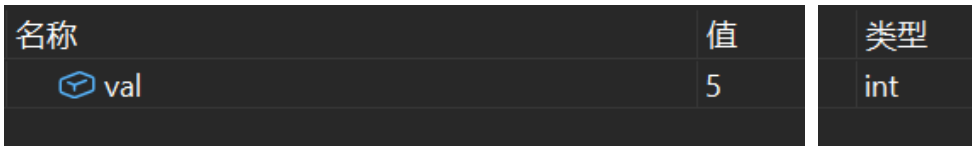
int fact(int val) {
    int result = 1;
    while (val > 1) {
        result *= val--;
    }
    return result;
}
```

逐次累乘并返回结果

图 1　阶乘函数与调用

main 调用 fact(5)并保存返回值。fact 用 result 累乘，同时将形参 val 逐次减小，最后返回阶乘值。





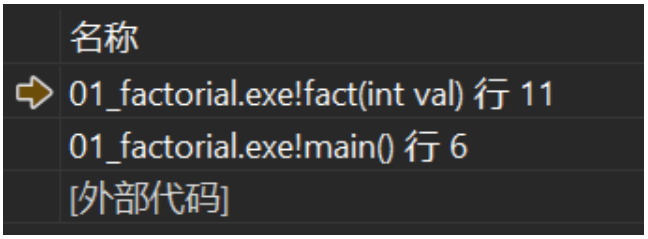
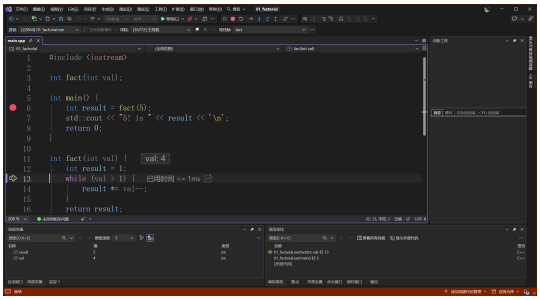
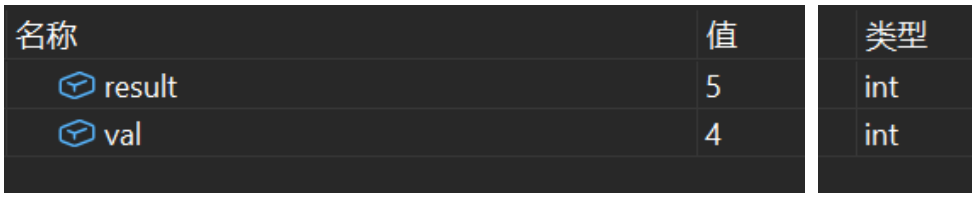


图 2　逐语句进入 fact 的停靠位置

进入 fact 后，val=5；调用堆栈显示调用来自 main 第 6 行。





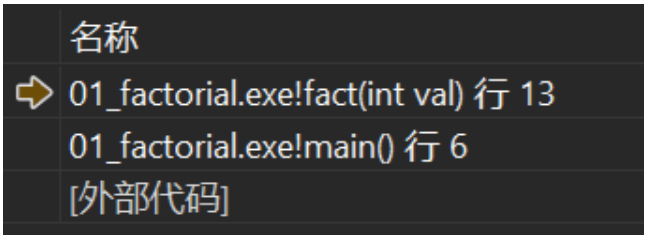
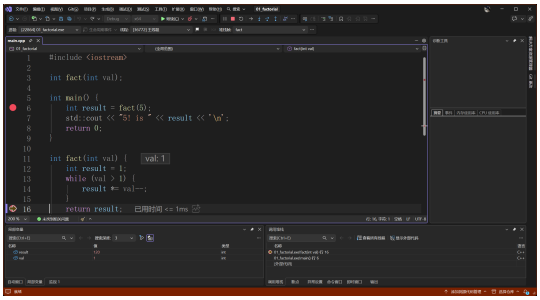


图 3　一次累乘后的 result 与 val

执行一次累乘后，result=5、val=4，程序回到 while 条件处。

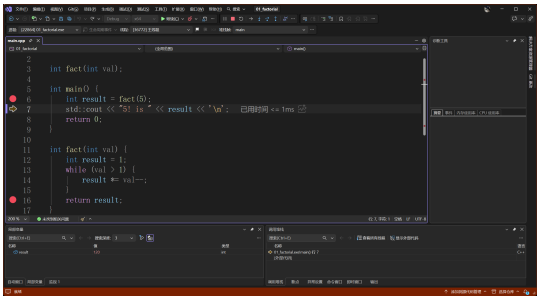


名称	值	类型
result	120	int
val	1	int

名称
01_factorial.exe!fact(int val) 行 16
01_factorial.exe!main() 行 6
[外部代码]

图 4 返回前的 result 与 val

循环结束，result=120、val=1，下一句返回 result。

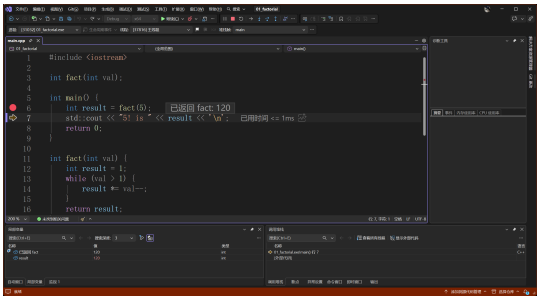


名称	值	类型
result	120	int

名称
01_factorial.exe!main() 行 7
[外部代码]

图 5 main 接收到返回值 120

返回 main 后，result 收到 120。fact 的局部变量随此次调用结束而离开作用域。



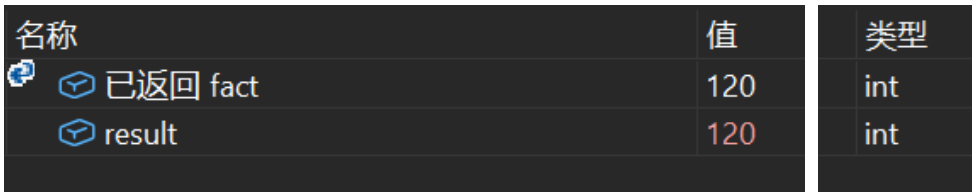
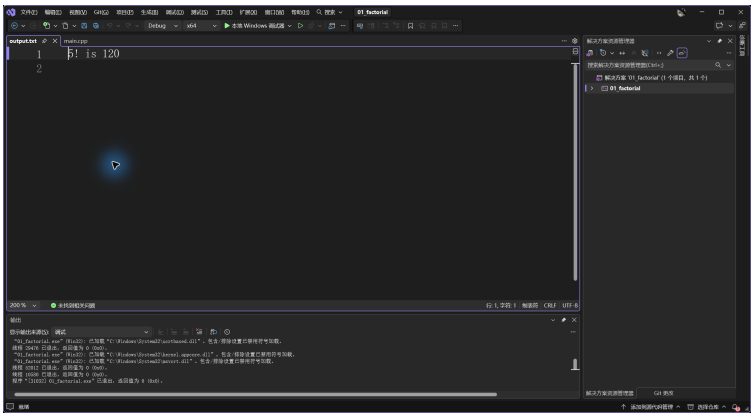




图 6　逐过程后的返回值与 result

在 main 的调用行按 F10 后直接停到第 7 行，result=120，未逐句进入 fact。



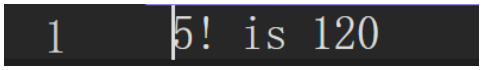


图 7　VS 运行生成的标准输出文件 output.txt

在 Visual Studio 中实际运行程序后，output.txt 中显示 5! is 120。这里展示的是标准输出重定向生成的文件内容。



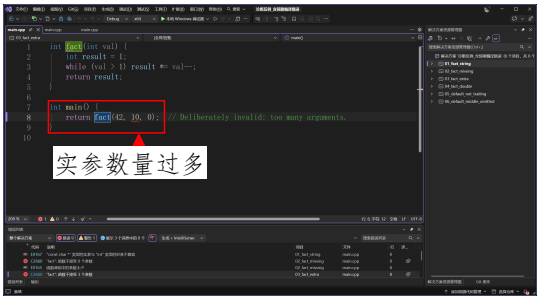
图 8　字符串实参的类型错误

字符串实参不能传给 int 形参，编译器给出 C2664。



图 9　遗漏必需实参的编译错误

fact 需要一个实参，空参数调用被编译器拒绝。

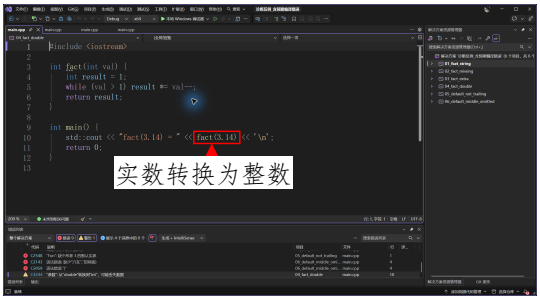



```
int fact(int val) {
    return fact(42, 10, 0);
```

C2660 "fact": 函数不接受 3 个参数

图 10 实参个数过多的编译错误

函数只接受一个参数，三个实参使构建失败。




```
int fact(int val) {
    std::cout << "fact(3.14) = " << fact(3.14) << '\n';
```

C4244 "参数": 从"double"转换到"int"，可能丢失数据

图 11 浮点实参转换时的编译警告

传入 double 实参时会进行到 int 的转换，可能丢失小数部分。该项目构建成功，但编译器给出 C4244 警告。

在 main 第 6 行按 F11 进入 fact，形参 val 为 5；第一次累乘后，result=5、val=4。

返回语句前，result=120、val=1；回到 main 第 7 行，main 中的 result 为 120。

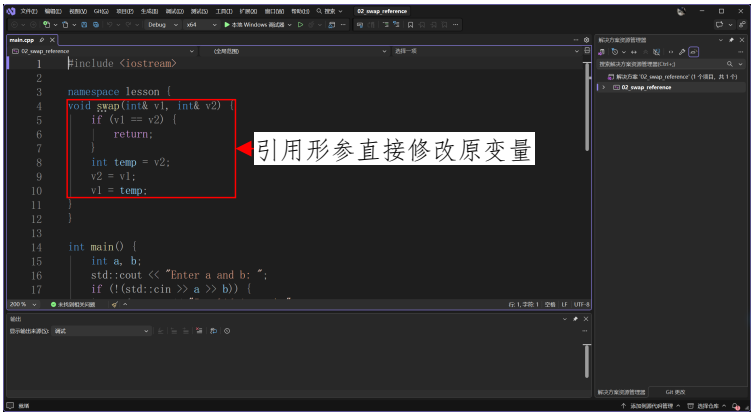
在同一调用行按 F10 逐过程，直接到 main 第 7 行，返回值和 result 均为 120。val 随该次 fact 调用结束而离开作用域，两个函数中的 result 各自独立。

四个调用变式的 MSVC 构建记录中，字符串实参报 C2664，缺少或多传实参报 C2660；实数实参产生 C4244 警告后可正常生成，fact(3.14)会先将实参转成 3，其阶乘为 6。

题目 2　课堂练习：引用参数交换

在 main 中输入两个整数，依次调用 swap(a,a) 和 swap(a,b)，单步观察两个调用的执行路径。

两个形参都是 int 引用。数值相等时直接返回，否则先用 temp 保存 v2，再完成两次赋值。



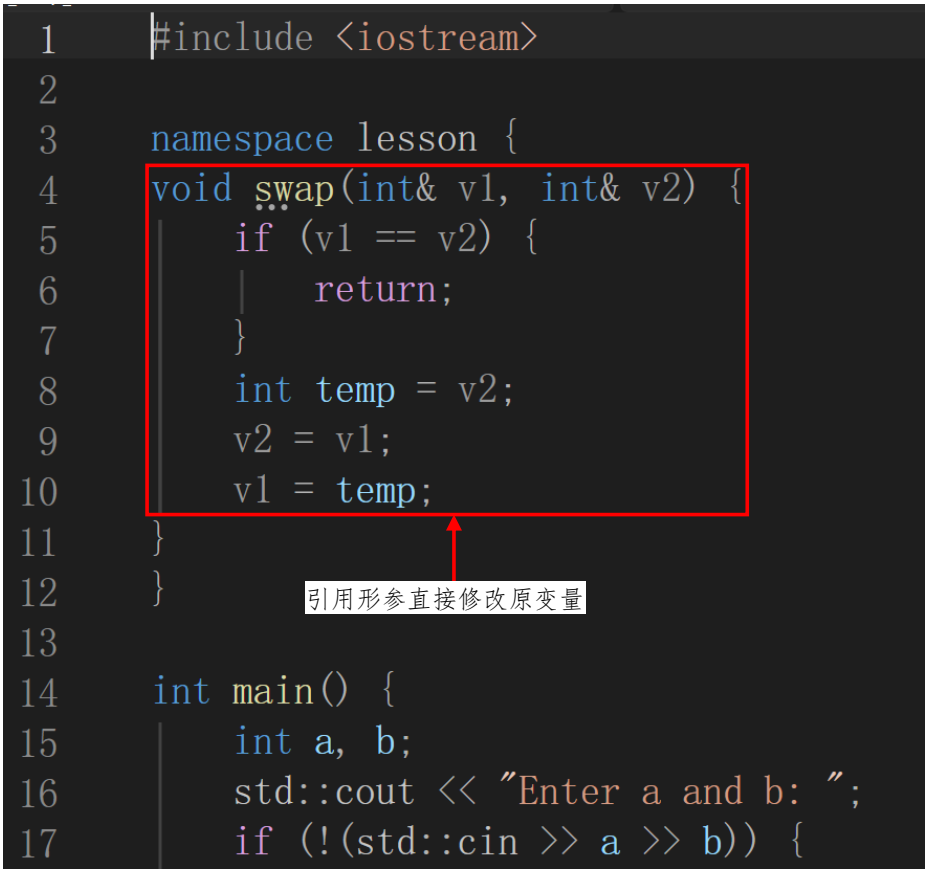
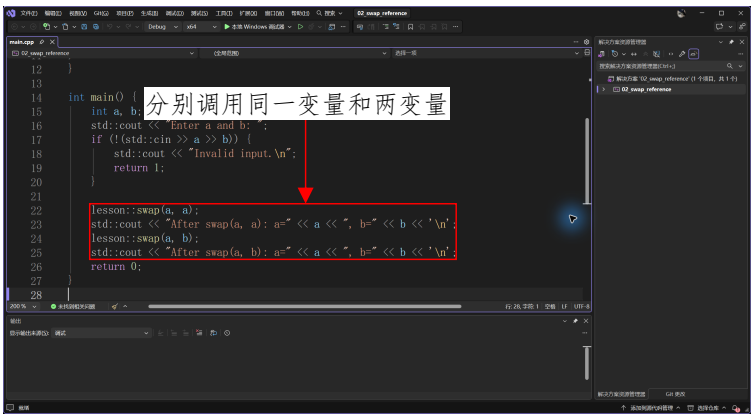


图 12　引用交换函数

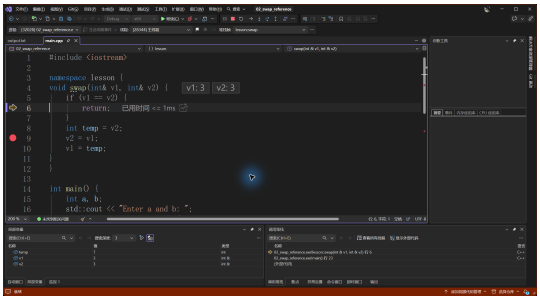
swap 接收两个引用。两值相同时直接返回，否则用 temp 暂存旧值，再完成两次赋值。




```
12  }
13
14  int main() {
15      int a, b;
16      std::cout << "Enter a and b: ";
17      if (!(std::cin >> a >> b)) {
18          std::cout << "Invalid input.\n";
19          return 1;
20      }
21
22      lesson::swap(a, a);
23      std::cout << "After swap(a, a): a=" << a << ", b=" << b << '\n';
24      lesson::swap(a, b);
25      std::cout << "After swap(a, b): a=" << a << ", b=" << b << '\n';
26      return 0;
27  }
```

图 13　两种引用交换调用

main 先调用 swap(a,a)，再调用 swap(a,b)，分别输出调用后的 a、b，便于比较提前返回与交换的区别。

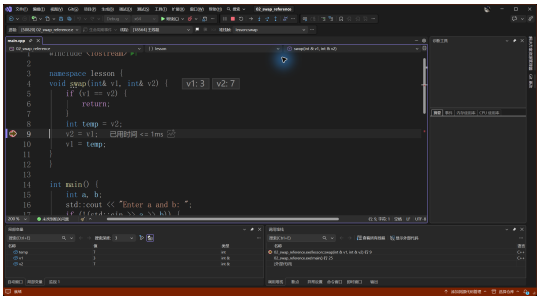


名称	值	类型
temp	7	int
v1	3	int &
v2	3	int &

名称
02_swap_reference.exe!lesson::swap(int & v1, int & v2) 行 6
02_swap_reference.exe!main() 行 23
[外部代码]

图 14　swap(a,a) 在判断后提前返回

v1、v2 均为 3，相等条件成立后直接返回；temp 尚未初始化，此显示值不参与判断。

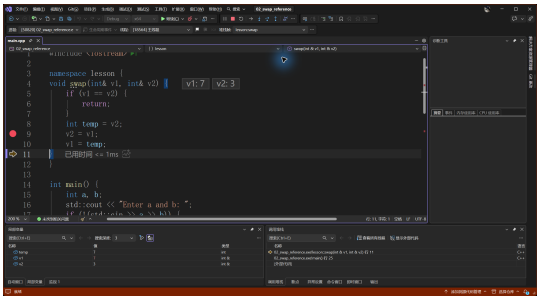


名称	值	类型
temp	7	int
v1	3	int &
v2	7	int &

名称
02_swap_reference.exe!lesson::swap(int & v1, int & v2) 行 9
02_swap_reference.exe!main() 行 25
[外部代码]

图 15　temp 保存旧值后的局部变量

temp 已保存 7，v1=3、v2=7；下一句才执行 v2=v1。

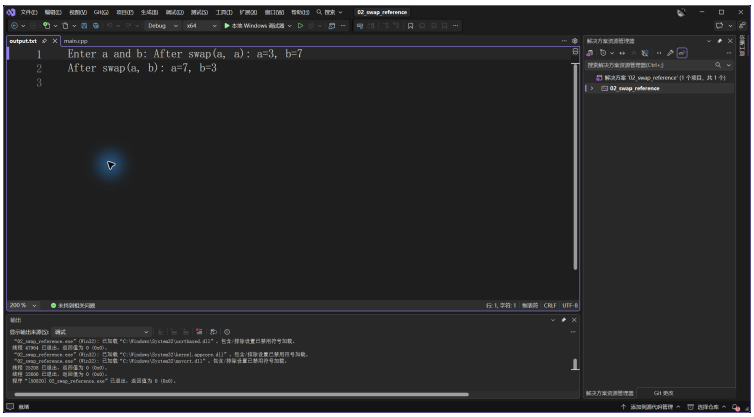


名称	值	类型
temp	7	int
v1	7	int &
v2	3	int &

名称
02_swap_reference.exe!lesson::swap(int & v1, int & v2) 行 11
02_swap_reference.exe!main() 行 25
[外部代码]

图 16　两次赋值完成后的局部变量

两次赋值结束，v1=7、v2=3，temp 仍为 7。



```
1    Enter a and b: After swap(a, a): a=3, b=7
2    After swap(a, b): a=7, b=3
```

图 17　VS 运行生成的 swap 标准输出文件

输入 3 和 7 时，同一变量调用后仍为 3、7；交换两个变量后变为 7、3。

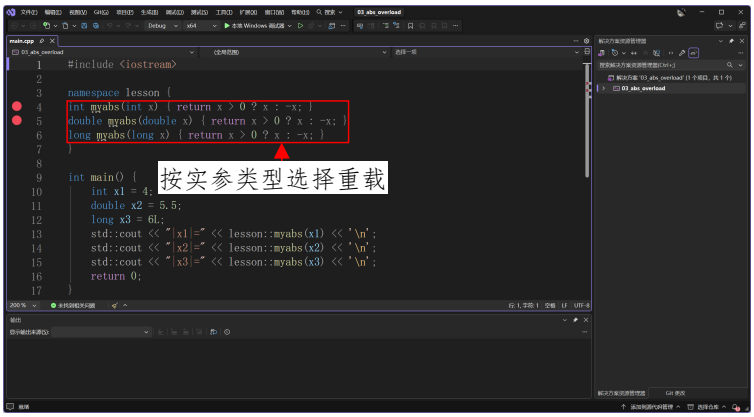
输入 3 和 7，swap(a,a)判断成立后直接返回，a、b 保持 3、7。

swap(a,b)先将 7 存入 temp，再完成两次赋值，最终 a=7、b=3。

题目 3　课堂练习：按参数类型重载

分别定义接收 int、double、long 的同名绝对值函数，观察三次调用选择的函数。

三个 lesson::myabs 重载的参数类型不同。main 中的 x1、x2、x3 分别使用 int、double 和 long。



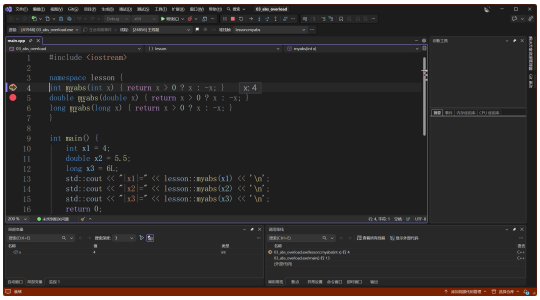

```cpp
#include <iostream>

namespace lesson {
int myabs(int x) { return x > 0 ? x : -x; }
double myabs(double x) { return x > 0 ? x : -x; }
long myabs(long x) { return x > 0 ? x : -x; }
}

int main() {
    int x1 = 4;
    double x2 = 5.5;
    long x3 = 6L;
    std::cout << "|x1|=" << lesson::myabs(x1) << '\n';
    std::cout << "|x2|=" << lesson::myabs(x2) << '\n';
    std::cout << "|x3|=" << lesson::myabs(x3) << '\n';
    return 0;
}
```

图 18 myabs 的三个重载

三个 myabs 函数分别接收 int、double 和 long。main 用对应类型的变量调用，函数名保持一致，重载由实参类型区分。

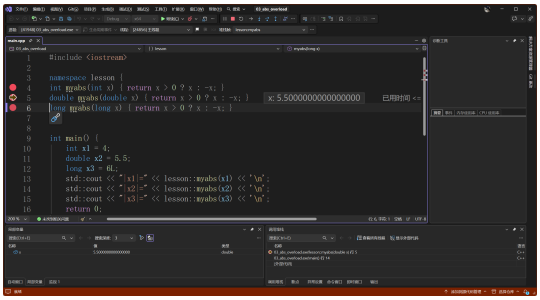


名称	值	类型
x	4	int



图 19 int 重载的形参和调用栈

x=4，类型为 int；调用堆栈对应 myabs(int)，由 main 第 13 行进入。当前停在第 4 行。



名称	值	类型
x	5.5000000000000000	double



图 20 double 重载的形参和调用栈

x=5.5，类型为 double；调用堆栈对应 myabs(double)，由 main 第 14 行进入。当前停在第 5 行。



图 21 long 重载的形参和调用栈

x=6，类型为 long；调用堆栈对应 myabs(long)，由 main 第 15 行进入。当前停在第 6 行。

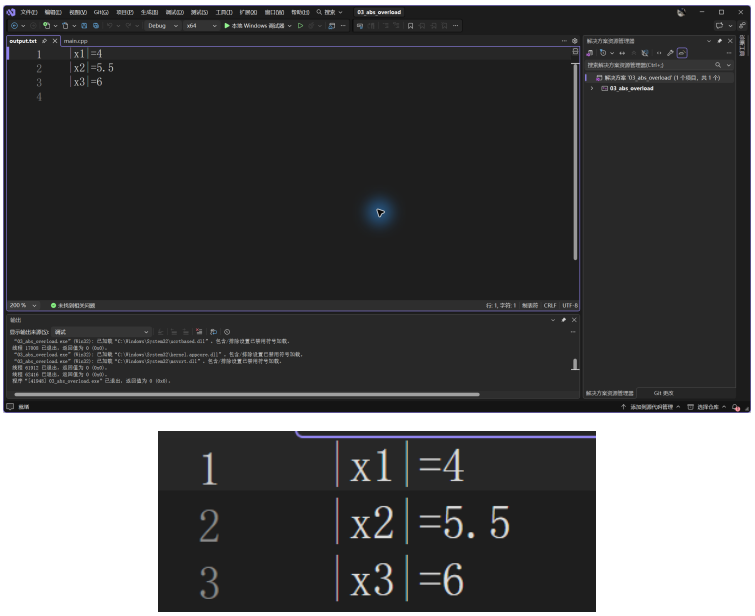


图 22 VS 运行生成的 myabs 标准输出文件

Visual Studio 继续执行并正常退出后，output.txt 中显示 4、5.5、6，与三次调用的结果一致。

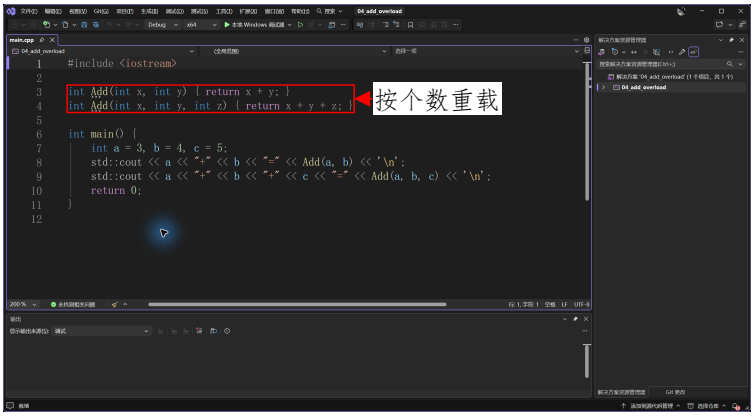
三个断点依次进入 myabs(int x)、myabs(double x)、myabs(long x)，对应形参值为 4、5.5、6。

重载由函数名和实参列表确定。若形参列表相同而仅改变返回类型，同一次调用就无法区分目标，因此不能仅靠返回类型构成重载。

题目 4　课堂练习：按参数个数重载

定义两参数和三参数两个 Add 重载，观察两次调用进入的函数。

Add(int,int) 返回两数之和，Add(int,int,int) 返回三数之和，main 根据实参数量调用对应版本。




```cpp
#include <iostream>

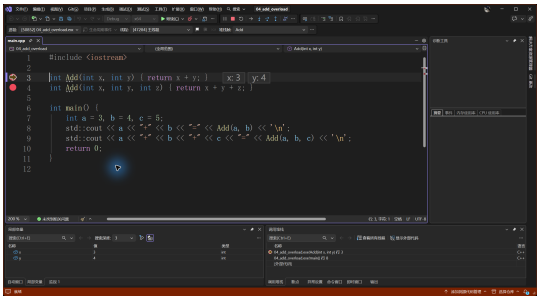
int Add(int x, int y) { return x + y; }
int Add(int x, int y, int z) { return x + y + z; }

int main() {
    int a = 3, b = 4, c = 5;
    std::cout << a << "+" << b << "=" << Add(a, b) << '\n';
    std::cout << a << "+" << b << "+" << c << "=" << Add(a, b, c) << '\n';
    return 0;
}
```

按实参个数选择重载

图 23　按参数个数区分 Add 重载

两个 Add 函数分别接收两个和三个整数。main 分别传入 a、b 以及 a、b、c，比较两次调用的结果。

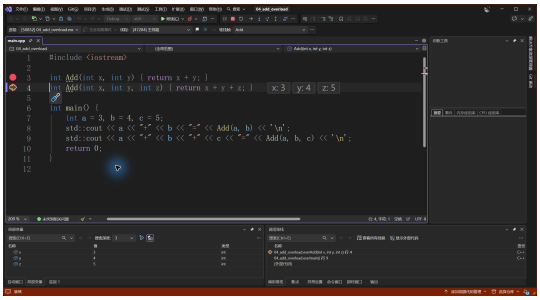


名称	值	类型
x	3	int
y	4	int

名称
04_add_overload.exe!Add(int x, int y) 行 3
04_add_overload.exe!main() 行 8
[外部代码]

图 24　两参数 Add 的局部变量

两参数重载的 x=3、y=4，调用堆栈对应 Add(int,int)。

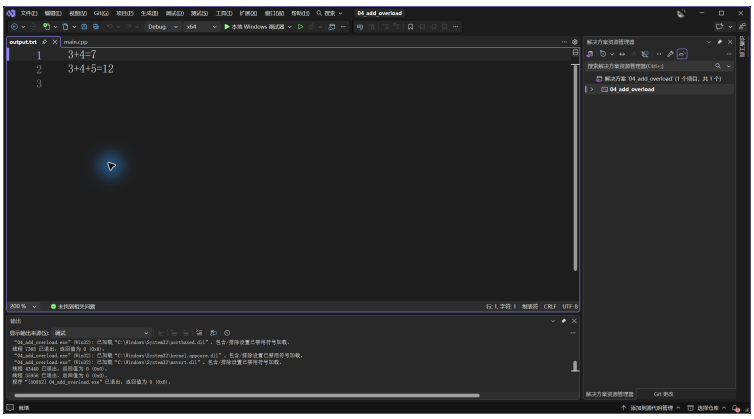


名称	值	类型
x	3	int
y	4	int
z	5	int

名称
04_add_overload.exe!Add(int x, int y, int z) 行 4
04_add_overload.exe!main() 行 9
[外部代码]

图 25　三参数 Add 的局部变量

三参数重载的 x=3、y=4、z=5，调用堆栈对应 Add(int,int,int)。



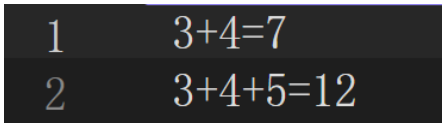


图 26 VS 运行生成的 Add 标准输出文件

两参数调用输出 3+4=7，三参数调用输出 3+4+5=12。

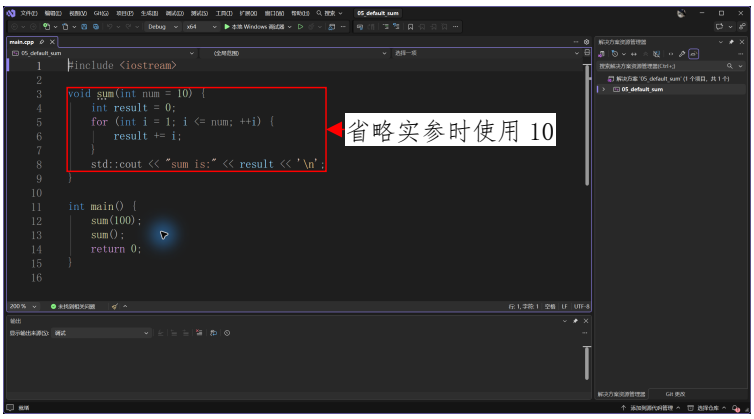
两参数调用进入 Add(int x,int y)，局部形参为 3、4，输出 7。

三参数调用进入 Add(int x,int y,int z)，局部形参为 3、4、5，输出 12。

题目 5 课堂练习：默认参数

调用 sum(100) 和 sum()，观察 num 的取值，并判断默认参数声明和省略实参的两个反例。

sum 的默认参数为 10，循环累加 1 至 num；提供实参时使用实参值，省略时使用调用点可见的默认值。




```cpp
#include <iostream>

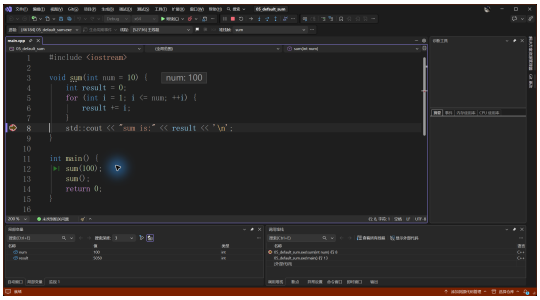
void sum(int num = 10) {
    int result = 0;
    for (int i = 1; i <= num; ++i) {
        result += i;
    }
    std::cout << "sum is:" << result << '\n';
}

int main() {
    sum(100);
    sum();
    return 0;
}
```

省略实参时使用 10

图 27　默认参数求和与调用

sum 把默认形参设为 10，从 1 累加到 num。main 分别调用 sum(100)和 sum()，对照显式实参与默认实参。

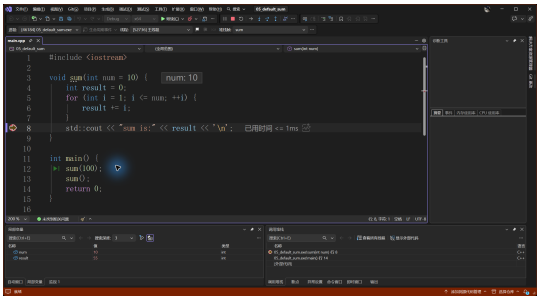


名称	值	类型
num	100	int
result	5050	int

名称
05_default_sum.exe!sum(int num) 行 8
05_default_sum.exe!main() 行 13
[外部代码]

图 28 显式实参求和后的 num 与 result

sum(100)这一轮的 num=100、result=5050，下一句输出结果。

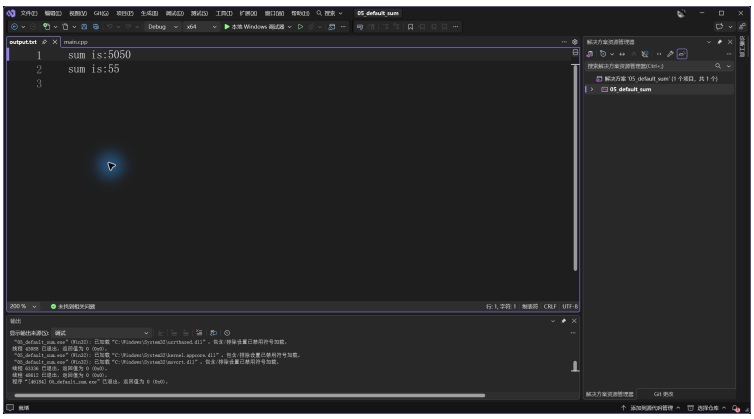


名称	值	类型
num	10	int
result	55	int

名称
05_default_sum.exe!sum(int num) 行 8
05_default_sum.exe!main() 行 14
[外部代码]

图 29 默认实参求和后的 num 与 result

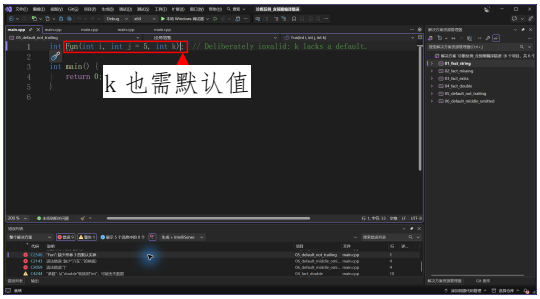
省略实参的 sum()中，num 取默认值 10，累加结果为 55。




```
1    sum is:5050
2    sum is:55
```

图 30 VS 运行生成的 sum 标准输出文件

sum(100) 输出 5050，省略实参的 sum() 输出 55。



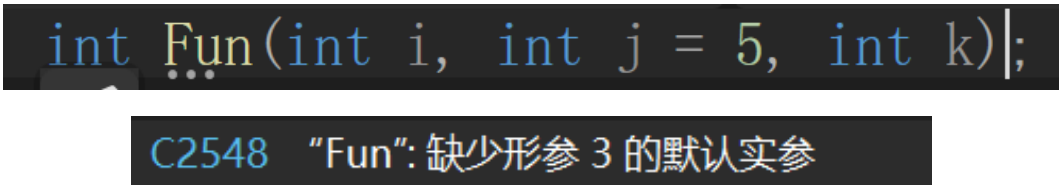
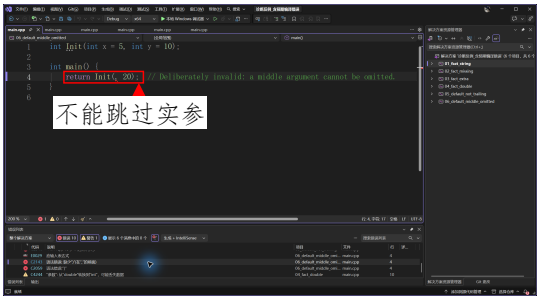


图 31 默认参数位置的编译错误

j 设置默认值后，后面的 k 也必须有默认值，因此此声明被拒绝。




```
int Init(int x = 5, int y = 10);
```

```
    return Init(, 20);
```

```
C2143  语法错误: 缺少")"(在","的前面)
C2059  语法错误:")"
```

图 32　中间实参留空的编译错误

调用不能用空位跳过前面的实参，Init(,20)触发语法错误。

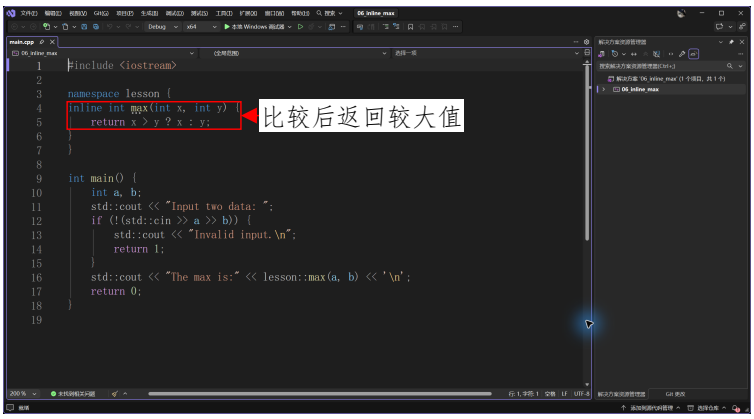
sum(100)中 num=100、result=5050；sum()中 num=10、result=55。显式实参覆盖默认值，省略实参时使用默认值 10。

默认值须在调用前可见；同一作用域中可以在后续声明或定义处补充此前未给的默认值，但同一形参不能重复指定。若前面的声明已给出默认值，定义处不再重复。

题目 6　课堂练习：内联函数求最大值

使用 inline 函数求两个整数的较大值，输入输出由 main 完成。

lesson::max 用条件运算符返回较大值，两个数相等时返回的数值不变。




```cpp
#include <iostream>

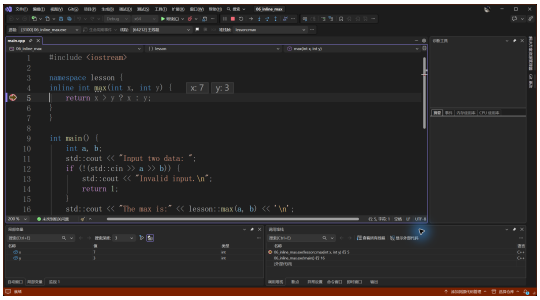
namespace lesson {
inline int max(int x, int y) {
    return x > y ? x : y;
}
}

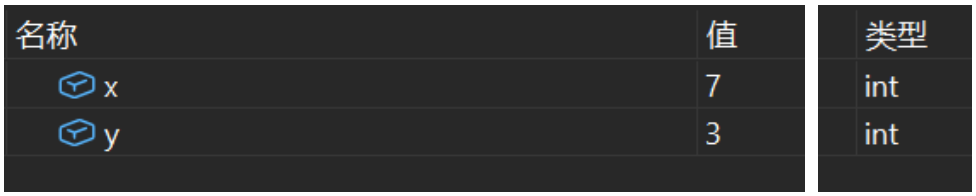
int main() {
    int a, b;
    std::cout << "Input two data: ";
    if (!(std::cin >> a >> b)) {
        std::cout << "Invalid input.\n";
        return 1;
    }
    std::cout << "The max is:" << lesson::max(a, b) << '\n';
    return 0;
}
```

比较后返回较大值

图 33　inline 最大值函数与调用

内联 max 用条件表达式返回两个整数中较大的一个。main 读取并检查输入，再显示函数返回值。





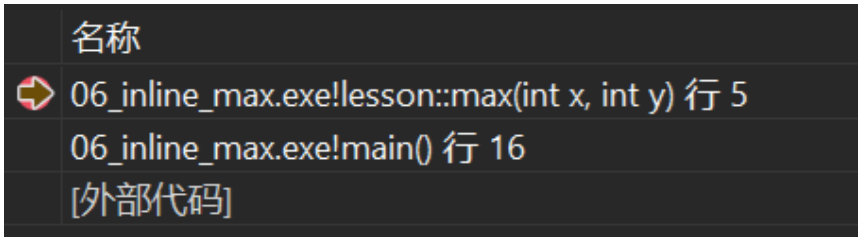
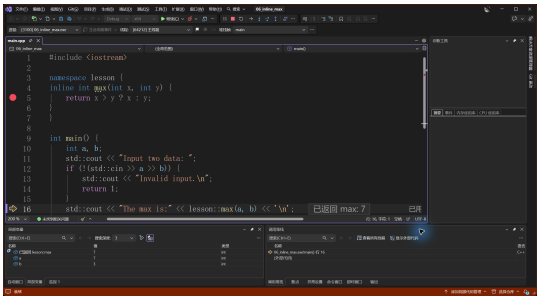


图 34 max 函数中的局部形参

max 中 x=7、y=3，当前将返回较大值。



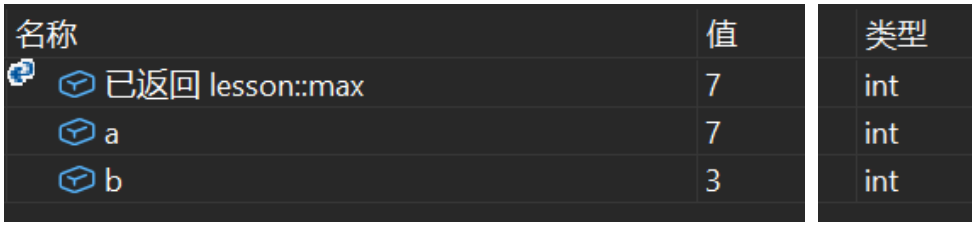
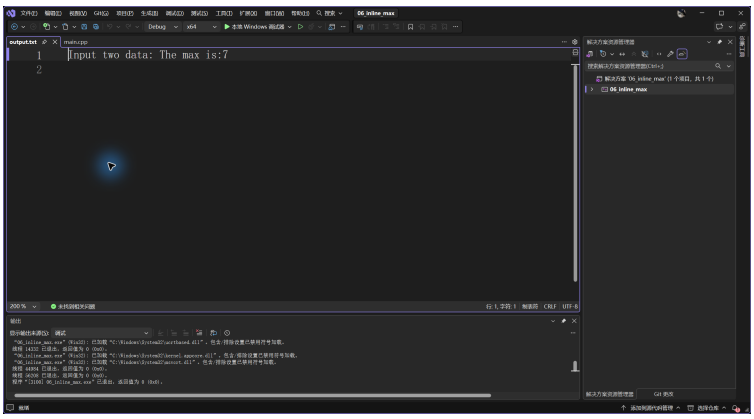




图 35 max 返回后的自动窗口

回到 main 时，自动窗口显示 max 返回 7；a 和 b 仍是 7、3。



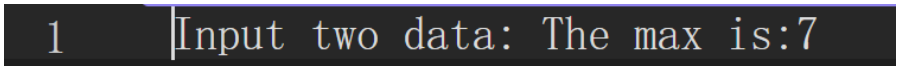


图 36 VS 运行生成的 max 标准输出文件

输入 7 和 3 后，实际生成的 output.txt 显示 The max is:7。

输入 7 和 3 后，max 中的形参 x=7、y=3。

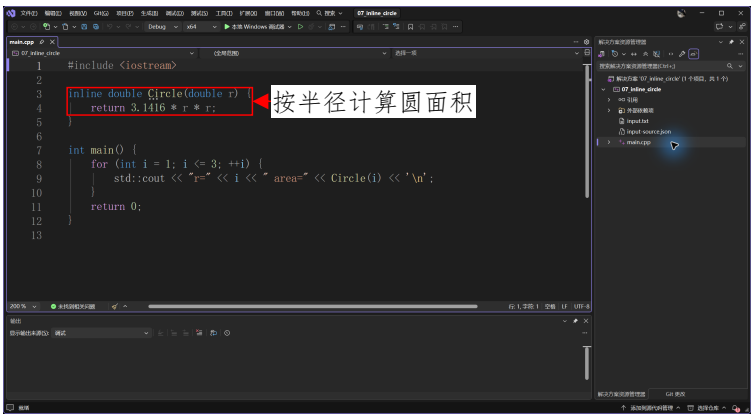
跳出函数后，返回值为 7，执行位置回到 main；output.txt 也显示最大值 7。

inline 关键字不保证函数一定展开，是否展开由编译器决定；本次 Debug 调试仍可进入函数体。

题目 7 课堂练习：内联函数计算圆面积

使用 inline 函数计算圆面积，输出半径为 1、2、3 时的结果。

Circle 使用课件中的近似值 3.1416 计算面积，main 通过循环依次传入半径。




```cpp
#include <iostream>

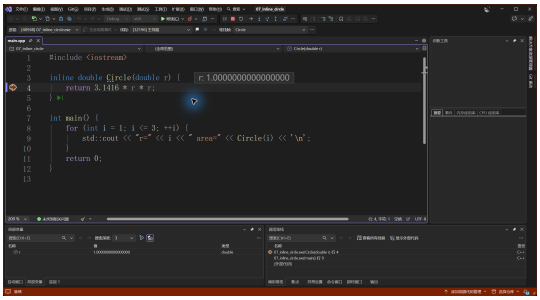
inline double Circle(double r) {
    return 3.1416 * r * r;
}

int main() {
    for (int i = 1; i <= 3; ++i) {
        std::cout << "r=" << i << " area=" << Circle(i) << '\n';
    }
    return 0;
}
```

按半径计算圆面积

图 37 Circle 函数与循环调用

Circle 返回 3.1416 乘半径的平方。main 用循环依次传入 1、2、3，输出各个半径对应的面积。





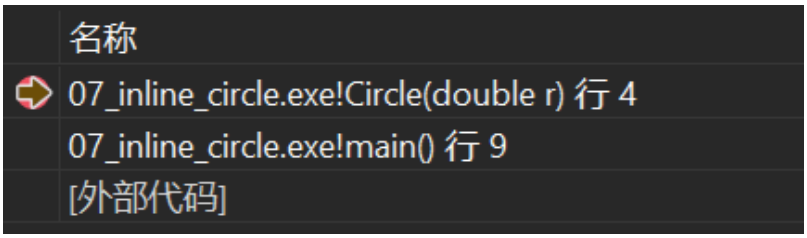
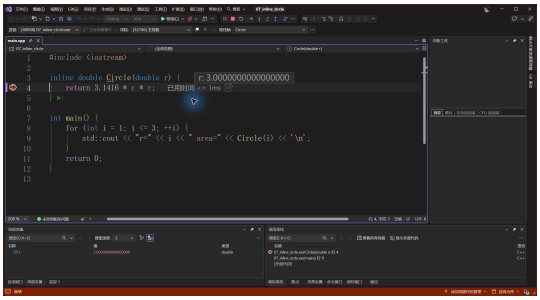


图 38 半径 r=1 的值与类型列

第一次进入 Circle，r=1，类型为 double。





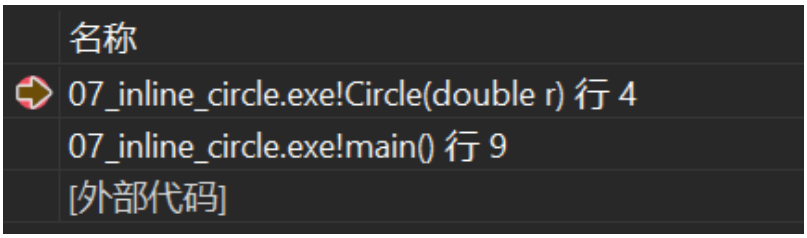


图 39 半径 r=3 的值与类型列

第三次进入 Circle，r=3，沿用同一个函数。

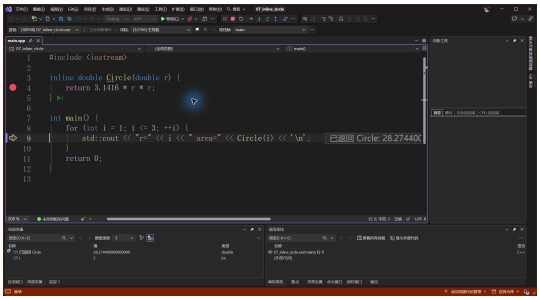
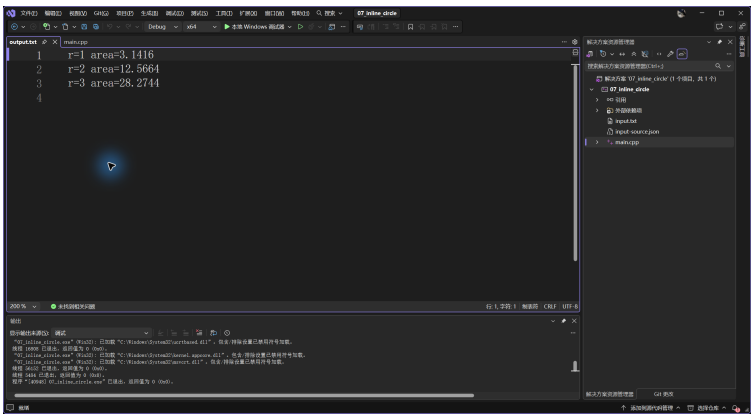






图 40 Circle 返回后的自动窗口

i=3 时，Circle 返回 28.2744，自动窗口中能看到该返回值。




```
1    r=1 area=3.1416
2    r=2 area=12.5664
3    r=3 area=28.2744
```

图 41 VS 运行生成的 Circle 标准输出文件

output.txt 中依次显示半径 1、2、3 的面积 3.1416、12.5664、28.2744。

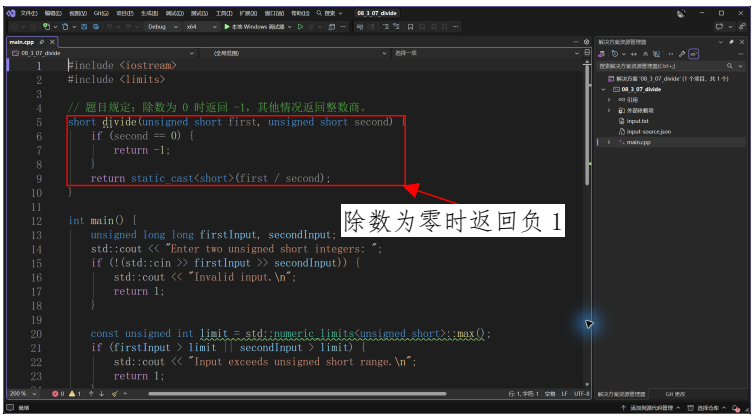
在同一断点依次观察到 r=1、2、3，均为 double。

i=3 时，Circle 返回 28.2744；三次面积输出依次为 3.1416、12.5664、28.2744。

题目 8 教材 3-7：两个无符号短整数相除

函数接收两个 unsigned short int 参数，返回 short int 型整数商；除数为零时返回 -1，在 main 中输入和输出。

main 先检查输入范围。除数非零时还检查整数商能否由 short 表示，再调用题目规定签名的 divide。




```cpp
#include <iostream>
#include <limits>

// 题目规定：除数为 0 时返回 -1，其他情况返回整数商。
short divide(unsigned short first, unsigned short second) {
    if (second == 0) {
        return -1;
    }
    return static_cast<short>(first / second);
}

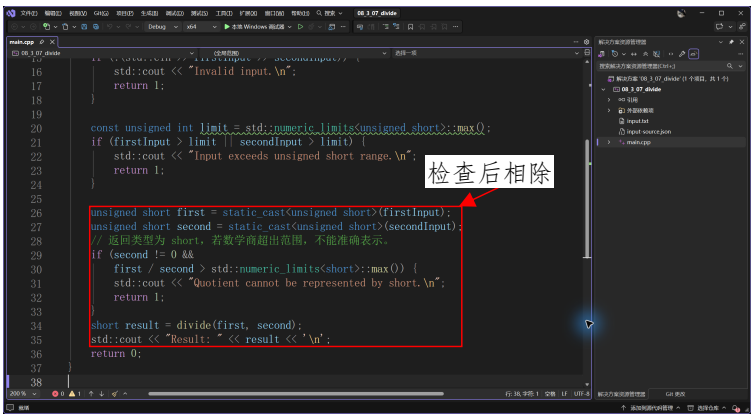
int main() {
    unsigned long long firstInput, secondInput;
    std::cout << "Enter two unsigned short integers: ";
    if (!(std::cin >> firstInput >> secondInput)) {
        std::cout << "Invalid input.\n";
        return 1;
    }

    const unsigned int limit = std::numeric_limits<unsigned short>::max();
    if (firstInput > limit || secondInput > limit) {
        std::cout << "Input exceeds unsigned short range.\n";
        return 1;
```

除数为零时返回负 1

图 42 divide 函数与输入检查

divide 的两个形参为 unsigned short，返回类型为 short；除数为 0 时返回-1。main 先检查输入是否成功及输入范围。

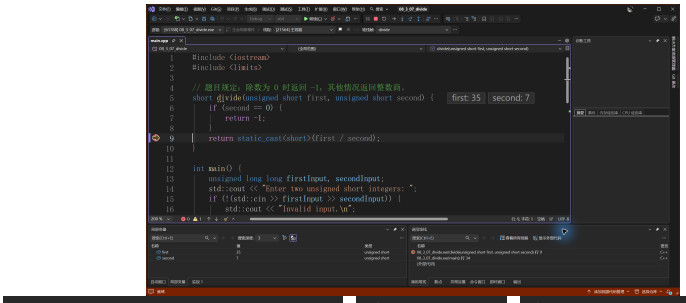



```cpp
        std::cout << "Invalid input.\n";
        return 1;
    }

    const unsigned int limit = std::numeric_limits<unsigned short>::max();
    if (firstInput > limit || secondInput > limit) {
        std::cout << "Input exceeds unsigned short range.\n";
        return 1;
    }
                                检查商的范围后调用
    unsigned short first = static_cast<unsigned short>(firstInput);
    unsigned short second = static_cast<unsigned short>(secondInput);
    // 返回类型为 short，若数学商超出范围，不能准确表示。
    if (second != 0 &&
        first / second > std::numeric_limits<short>::max()) {
        std::cout << "Quotient cannot be represented by short.\n";
        return 1;
    }
    short result = divide(first, second);
    std::cout << "Result: " << result << '\n';
    return 0;
}
```

图 43　范围检查、整数商与输出

输入通过范围检查后再转为 unsigned short。除数非零时检查数学商能否用 short 表示，通过后调用 divide 并输出结果。

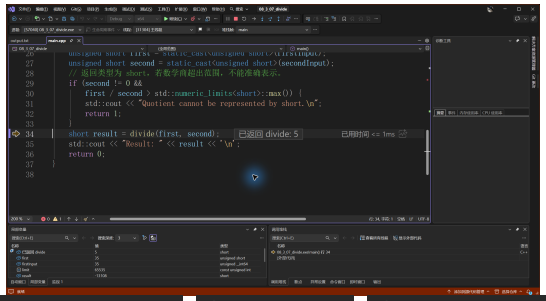


名称	值	类型
first	35	unsigned short
second	7	unsigned short

名称
08_3_07_divide.exe!divide(unsigned short first, unsigned short second) 行 9
08_3_07_divide.exe!main() 行 34
[外部代码]

图 44　divide 两个形参的值与类型列

divide 接收到 35 和 7，两个形参类型均为 unsigned short。



名称	值	类型
已返回 divide	5	short
first	35	unsigned short
firstInput	35	unsigned __int64
limit	65535	const unsigned int
result	-13108	short

名称
08_3_07_divide.exe!main() 行 34
[外部代码]

图 45　divide 返回值 5 及 short 类型

divide 返回 5，类型为 short；result 尚未完成赋值，不采用其未初始化显示值。

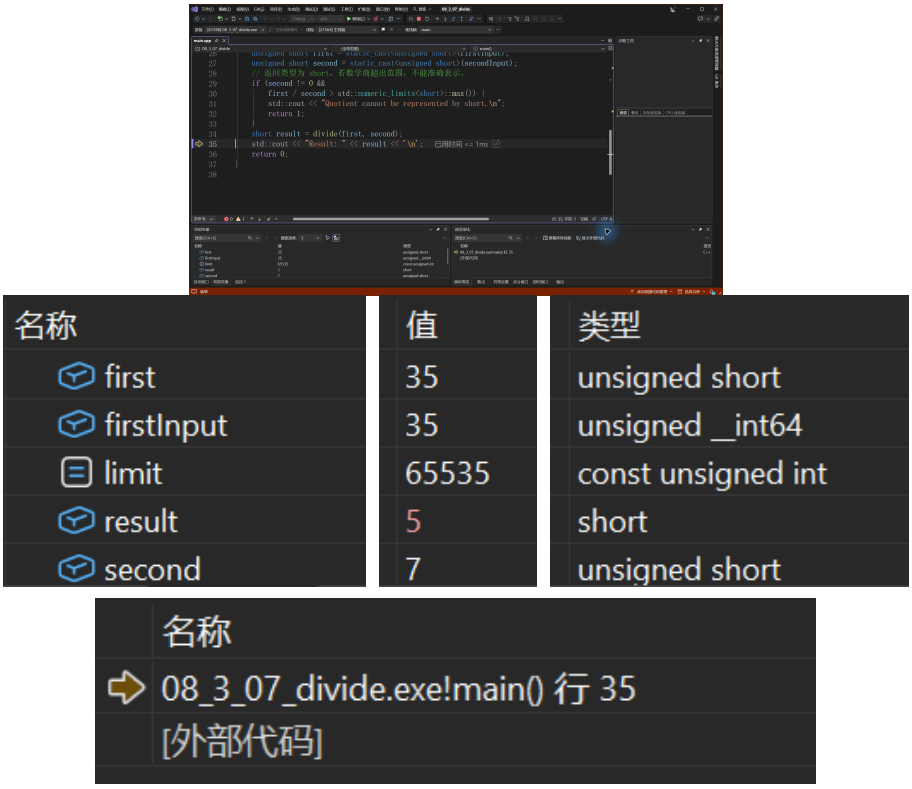
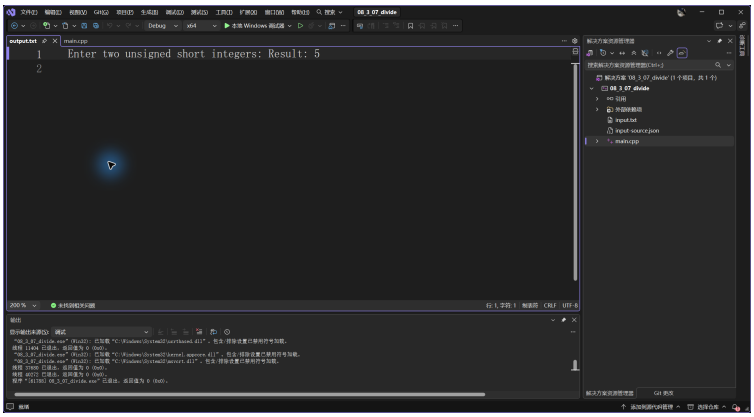


图 46 F10 赋值后 main 中的 result

再按 F10 完成赋值，result=5；first=35、second=7 保持不变。



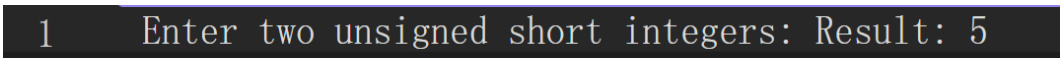


图 47 VS 运行生成的 divide 标准输出文件

以 35 和 7 为样例运行后，生成的 output.txt 中显示 Result: 5。

进入 divide 后，first=35、second=7，形参类型均为 unsigned short。

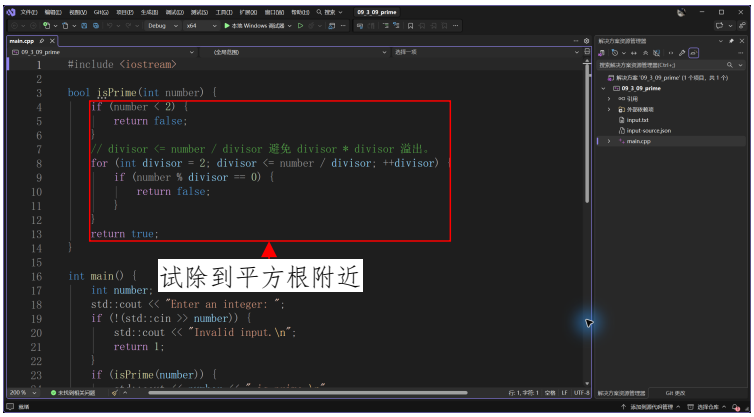
跳出函数到 main 第 34 行时，VS 显示返回值为 5、类型为 short；执行 F10 到第 35 行，result 接收到 5。

标准输出文件中显示 Result: 5。

题目 9 教材 3-9：判断质数

编写函数判断一个整数是否为质数，在 main 中输入并输出判断结果。

小于 2 的整数返回 false；其余数逐个检查可能的因子，发现整除因子就返回 false。循环条件写为 divisor <= number/divisor，避免乘法溢出。



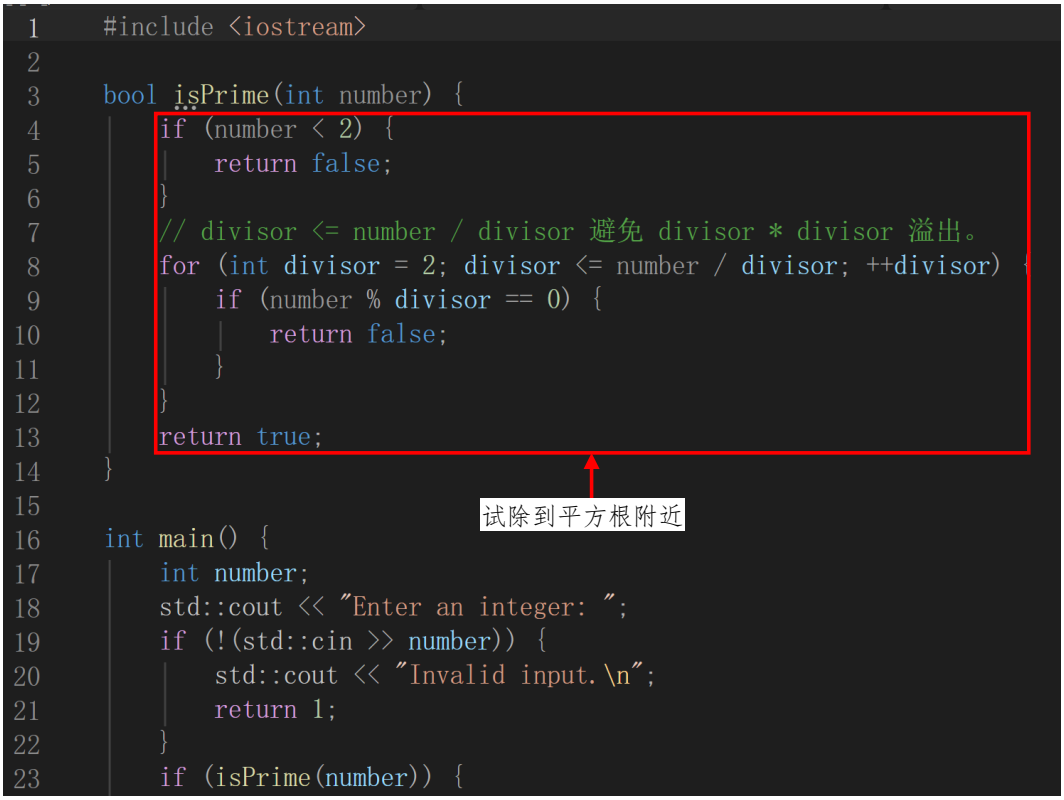
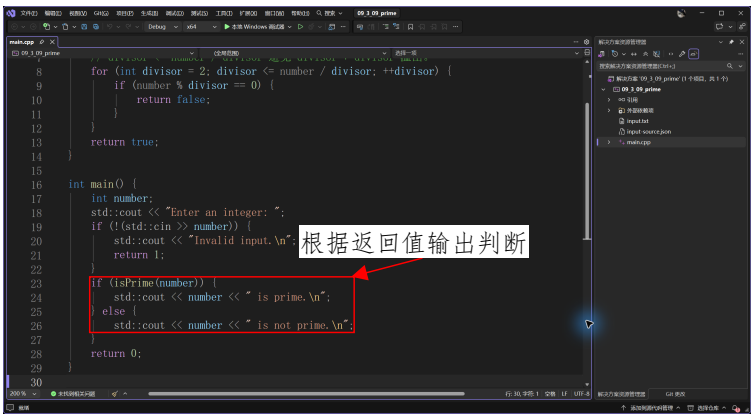


图 48 质数判断函数

isPrime 先排除小于 2 的数，再从 2 开始试除。循环条件写为 divisor<=number/divisor，避免直接计算平方产生溢出。

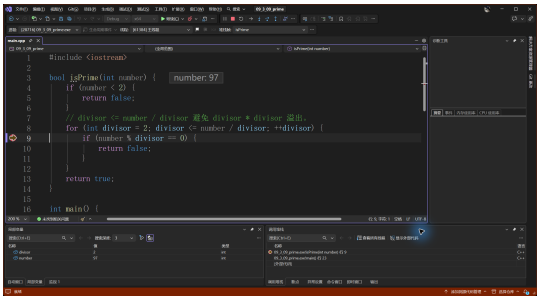



```cpp
    for (int divisor = 2; divisor <= number / divisor; ++divisor) {
        if (number % divisor == 0) {
            return false;
        }
    }
    return true;
}

int main() {
    int number;
    std::cout << "Enter an integer: ";
    if (!(std::cin >> number)) {
        std::cout << "Invalid input.\n";
        return 1;
    }
    if (isPrime(number)) {
        std::cout << number << " is prime.\n";
    } else {
        std::cout << number << " is not prime.\n";
    }
    return 0;
}
```


图 49　质数判断的输入与输出

main 读取整数并调用 isPrime，根据返回的 bool 值输出质数或非质数；输入失败时不继续判断。

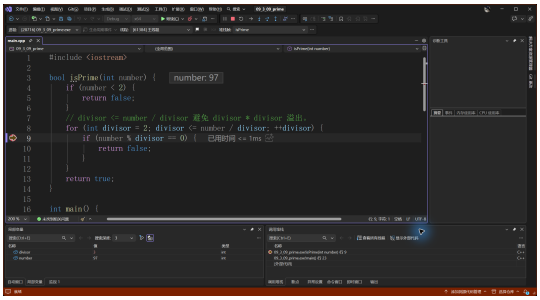


名称	值	类型
divisor	2	int
number	97	int

名称
09_3_09_prime.exe!isPrime(int number) 行 9
09_3_09_prime.exe!main() 行 23
[外部代码]

图 50　第一次断点中的 number 与 divisor

第一次停在取余判断处，number=97、divisor=2。

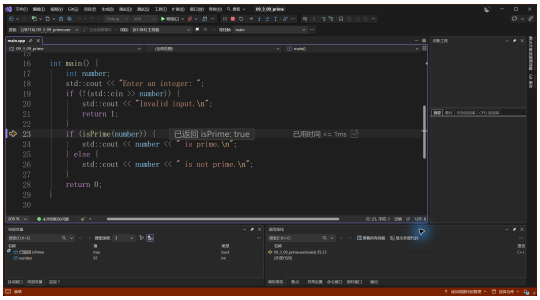


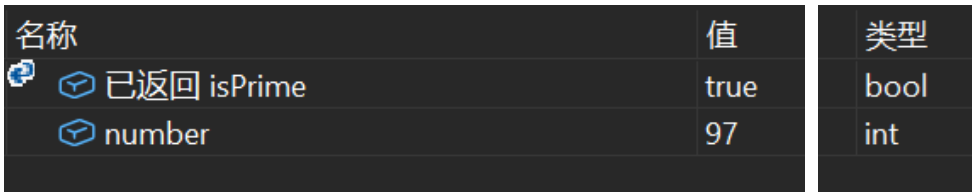
名称	值	类型
divisor	3	int
number	97	int

名称
09_3_09_prime.exe!isPrime(int number) 行 9
09_3_09_prime.exe!main() 行 23
[外部代码]

图 51　再次命中断点时的 divisor=3

再次命中同一断点时，divisor 增为 3，number 仍是 97。





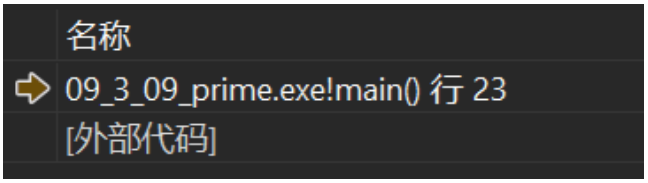


图 52 isPrime 返回后的自动窗口

返回 main 后，isPrime 的返回值为 true，表示 97 是质数。

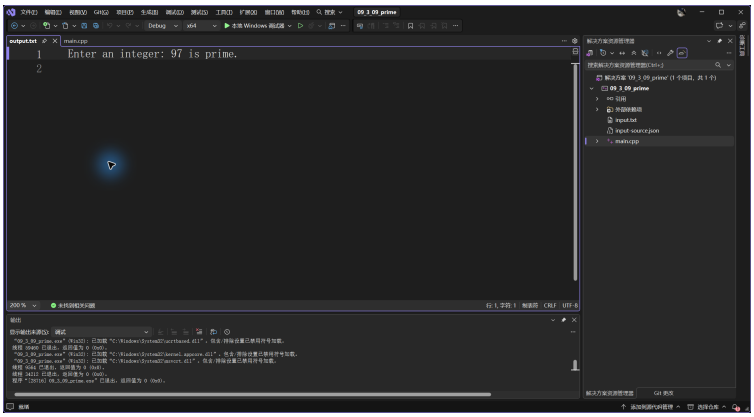




图 53 VS 运行生成的 prime 标准输出文件

output.txt 中显示“97 is prime.”，与本次返回值 true 一致。

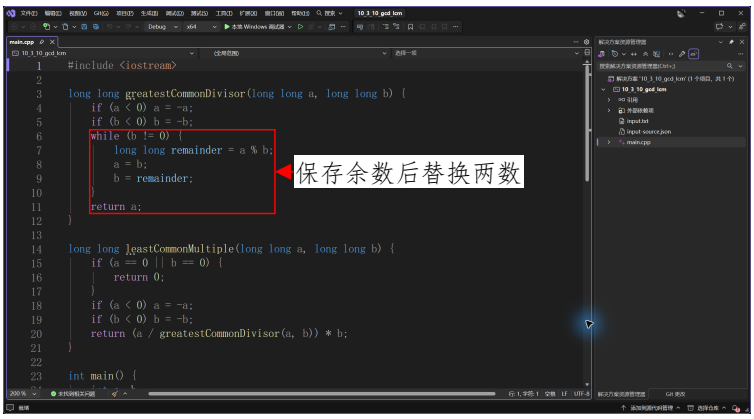
在第 9 行的两次断点中，number 保持 97，divisor 依次为 2、3。移除断点后跳出 isPrime，返回 true。

标准输出判断 97 为质数。

题目 10 教材 3-10：最大公约数和最小公倍数

分别编写函数求两个整数的最大公约数与最小公倍数，输入和结果输出放在 main 中。

最大公约数使用辗转相除法；最小公倍数先除以最大公约数再乘另一数。输入为 int，计算使用 long long，并处理负数与零。




```cpp
#include <iostream>

long long greatestCommonDivisor(long long a, long long b) {
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    while (b != 0) {
        long long remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}

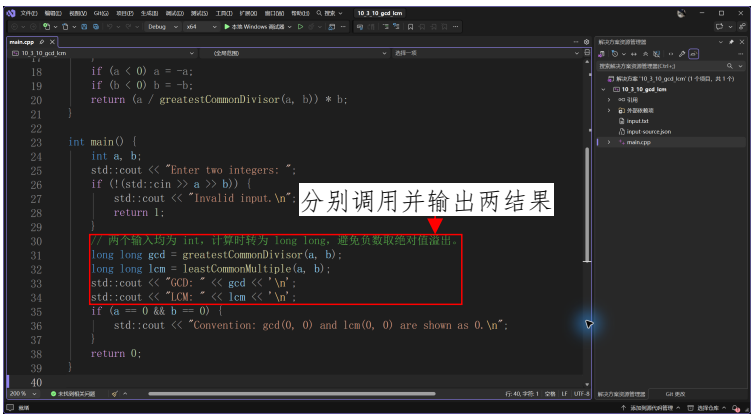
long long leastCommonMultiple(long long a, long long b) {
    if (a == 0 || b == 0) {
        return 0;
    }
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    return (a / greatestCommonDivisor(a, b)) * b;
}

int main() {
```

保存余数后替换两数

图 54　最大公约数与最小公倍数函数

最大公约数函数反复计算余数，再用 b 和余数替换 a、b。最小公倍数先除以最大公约数，再与另一数相乘。



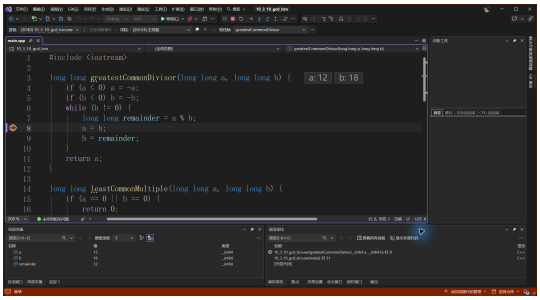

```cpp
    if (a < 0) a = -a;
    if (b < 0) b = -b;
    return (a / greatestCommonDivisor(a, b)) * b;
}

int main() {
    int a, b;
    std::cout << "Enter two integers: ";
    if (!(std::cin >> a >> b)) {
        std::cout << "Invalid input.\n";
        return 1;
    }
    // 两个输入均为 int，计算时转为 long long，避免负数取绝对值溢出。
    long long gcd = greatestCommonDivisor(a, b);
    long long lcm = leastCommonMultiple(a, b);
    std::cout << "GCD: " << gcd << '\n';
    std::cout << "LCM: " << lcm << '\n';
    if (a == 0 && b == 0) {
        std::cout << "Convention: gcd(0, 0) and lcm(0, 0) are shown as 0.\n";
    }
    return 0;
}
```

分别调用并输出两结果

图 55　公约数、公倍数的调用与输出

main 接收两个 int 输入，以 long long 计算公约数和公倍数并分别输出。两数均为 0 时另行说明程序采用的约定。

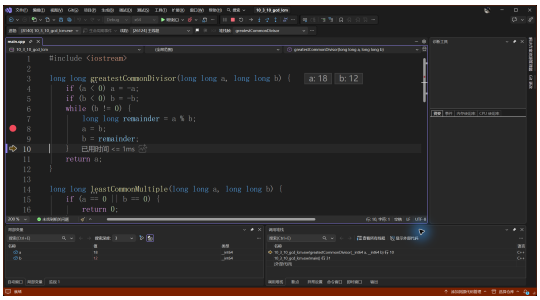


名称	值	类型
a	12	__int64
b	18	__int64
remainder	12	__int64

名称
10_3_10_gcd_lcm.exe!greatestCommonDivisor(__int64 a, __int64 b) 行 8
10_3_10_gcd_lcm.exe!main() 行 31
[外部代码]

图 56　计算余数后的局部变量

第一次求余后，a=12、b=18、remainder=12。



名称	值	类型
a	18	__int64
b	12	__int64

名称
10_3_10_gcd_lcm.exe!greatestCommonDivisor(__int64 a, __int64 b) 行 10
10_3_10_gcd_lcm.exe!main() 行 31
[外部代码]

图 57　完成两次替换后的 a 与 b

完成两次变量替换后，a=18、b=12，供下一轮求余使用。



图 58 greatestCommonDivisor 的返回值 6

greatestCommonDivisor 返回 6，下一步将它保存到 main 中的 gcd。



图 59 main 接收的公约数与公倍数

两个调用结束，gcd=6、lcm=36；自动窗口还显示最小公倍数函数返回 36。

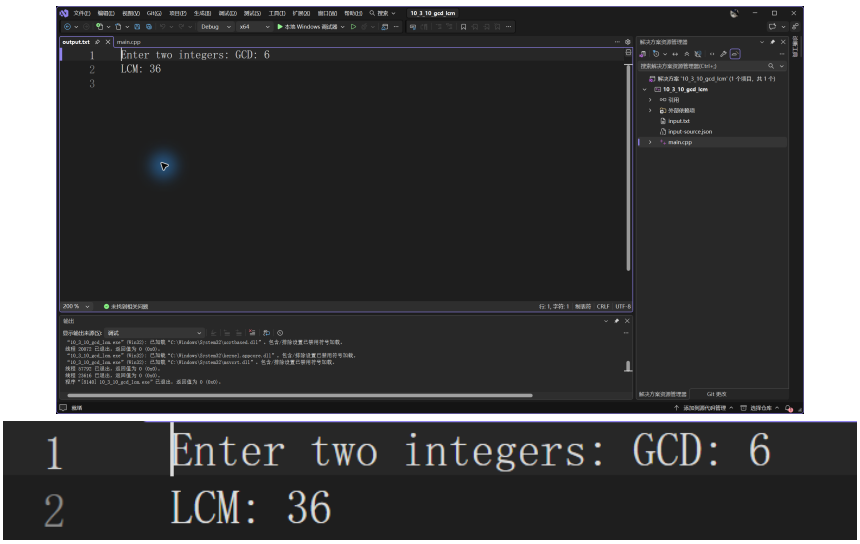


图 60 VS 运行生成的 gcd_lcm 标准输出文件

以 12 和 18 为样例运行后，output.txt 中输出 GCD: 6 和 LCM: 36。

计算余数后，a=12、b=18、remainder=12；执行两次替换后，a=18、b=12。

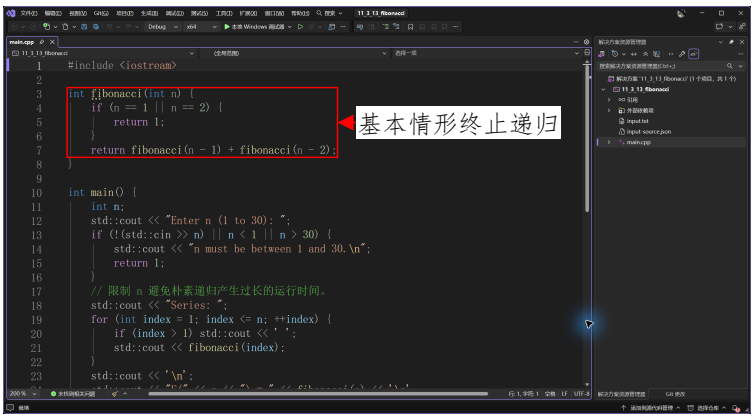
greatestCommonDivisor 返回 6，随后 main 接收到 gcd=6；完成第二个函数调用后，lcm=36。

标准输出分别为 GCD: 6、LCM: 36。

题目 11 教材 3-13：递归斐波那契级数

按 F1=F2=1、Fn=F(n-1)+F(n-2) 编写递归函数，并观察递归调用过程。

n 为 1 或 2 时返回 1，其余项递归求前两项之和。main 输出前 n 项和第 n 项，并把演示范围限制为 1 至 30。




```cpp
#include <iostream>

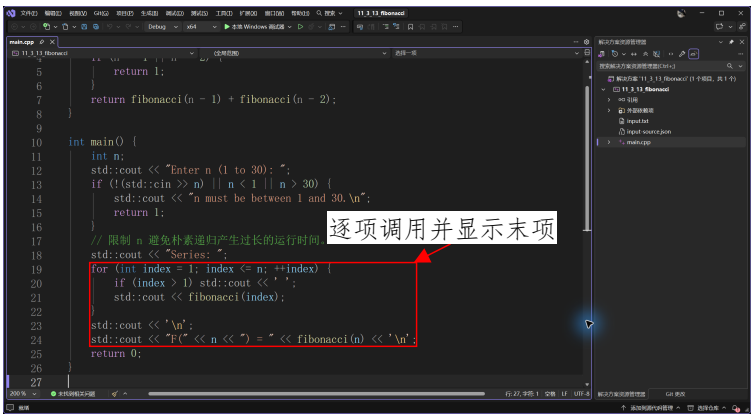
int fibonacci(int n) {
    if (n == 1 || n == 2) {
        return 1;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n;
    std::cout << "Enter n (1 to 30): ";
    if (!(std::cin >> n) || n < 1 || n > 30) {
        std::cout << "n must be between 1 and 30.\n";
        return 1;
    }
    // 限制 n 避免朴素递归产生过长的运行时间。
    std::cout << "Series: ";
    for (int index = 1; index <= n; ++index) {
        if (index > 1) std::cout << ' ';
        std::cout << fibonacci(index);
    }
    std::cout << '\n';
```

基本情形终止递归

图 61　斐波那契递归与输入检查

fibonacci 在 n 为 1 或 2 时返回 1，其余情况递归求前两项之和。main 把输入限制在 1 到 30，控制朴素递归的运行时间。

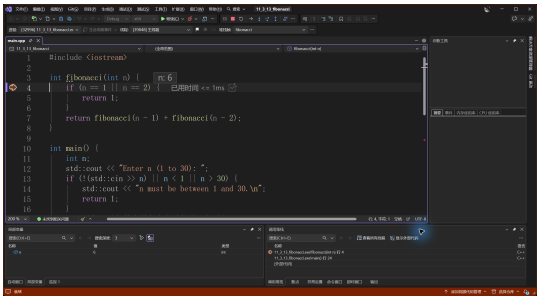


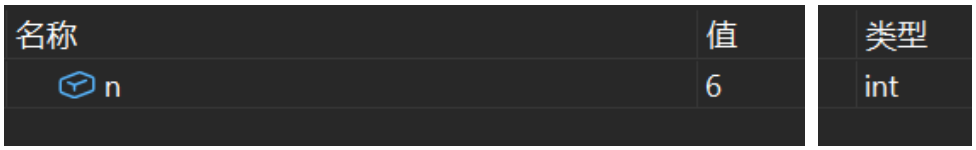

```cpp
        return 1;
    }
    return fibonacci(n - 1) + fibonacci(n - 2);
}

int main() {
    int n;
    std::cout << "Enter n (1 to 30): ";
    if (!(std::cin >> n) || n < 1 || n > 30) {
        std::cout << "n must be between 1 and 30.\n";
        return 1;
    }
    // 限制 n 避免朴素递归产生过长的运行时间。
    std::cout << "Series: ";
    for (int index = 1; index <= n; ++index) {
        if (index > 1) std::cout << ' ';
        std::cout << fibonacci(index);
    }
    std::cout << '\n';
    std::cout << "F(" << n << ") = " << fibonacci(n) << '\n';
    return 0;
}
```

图 62　数列输出与末项调用

循环依次调用 fibonacci 输出前 n 项，随后再显示第 n 项的值。序列和末项分别输出，便于核对。





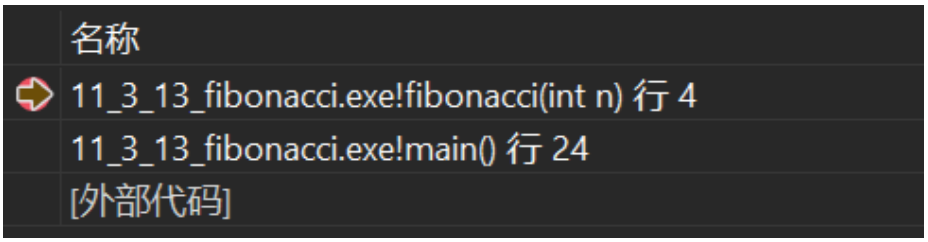
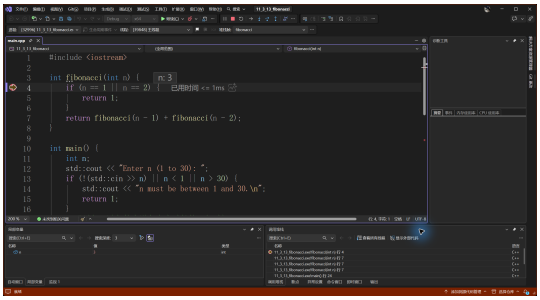
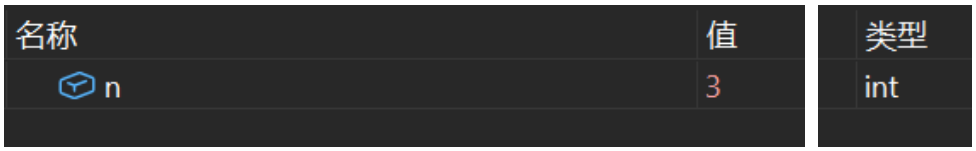


图 63　本次最终递归调用的 n=6

进入最后一次 fibonacci(n)调用，n=6；调用来自 main 第 24 行。





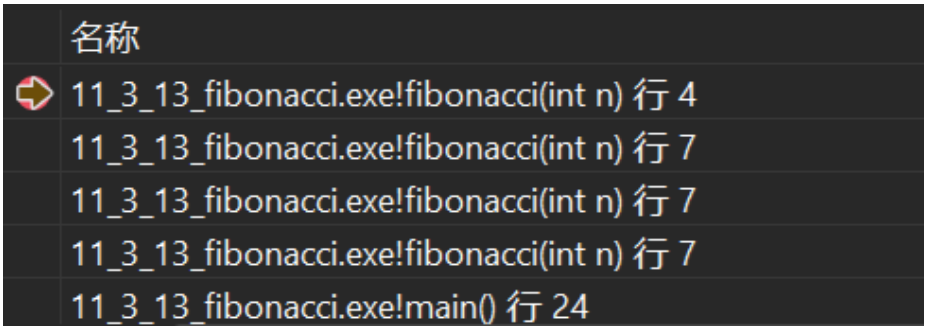
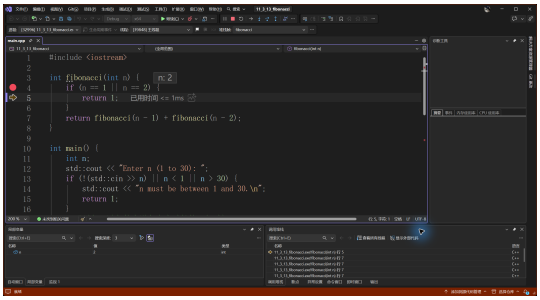


图 64　n=3 暂停点的多层递归调用堆栈

递归到 n=3 时，堆栈保留了上层各次 fibonacci 调用，最下方是 main 第 24 行。

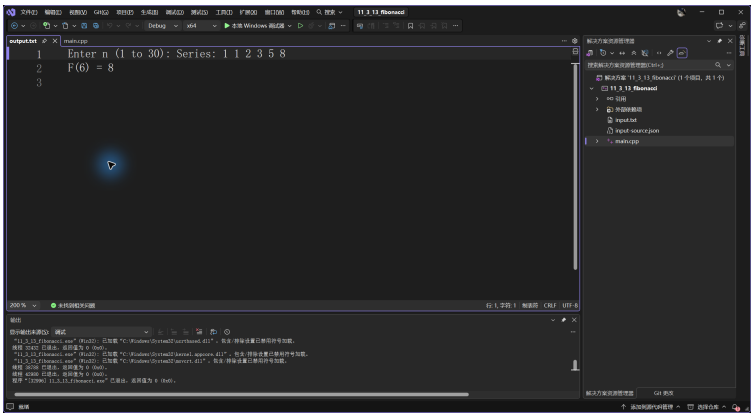


名称	值	类型
n	2	int

名称
11_3_13_fibonacci.exe!fibonacci(int n) 行 5
11_3_13_fibonacci.exe!fibonacci(int n) 行 7
11_3_13_fibonacci.exe!fibonacci(int n) 行 7
11_3_13_fibonacci.exe!fibonacci(int n) 行 7
11_3_13_fibonacci.exe!fibonacci(int n) 行 7

图 65　n=2 时停在基本情形 return 1

n=2 满足基本情形，当前停在 return 1，尚未执行返回。



```
1    Enter n (1 to 30): Series: 1 1 2 3 5 8
2    F(6) = 8
```

图 66　VS 运行生成的 fibonacci 标准输出文件

output.txt 中的序列为 1 1 2 3 5 8，并显示 F(6)=8。

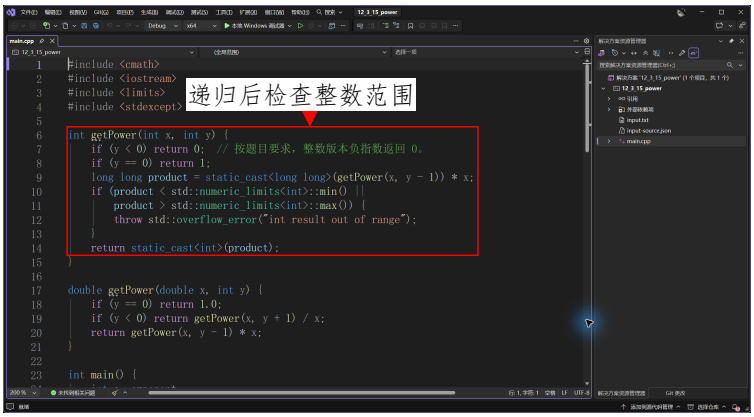
本次输入 n=6。继续命中递归断点后，n=3 时已形成多层调用；n=2 进入基本情形，准备返回 1。

程序输出前 6 项 1 1 2 3 5 8，第 6 项为 8。

题目 12　教材 3-15：递归重载求幂

在同一程序中实现两个递归 getPower 重载，分别接收 int 和 double 底数；整型版本在负指数时返回 0。

两个重载都以指数为 0 为终止条件。实数版本的负指数逐层加 1，并在返回时除以底数。要让整数 a 调用实数重载，使用 static_cast<double>(a)。




```cpp
#include <cmath>
#include <iostream>
#include <limits>
#include <stdexcept>

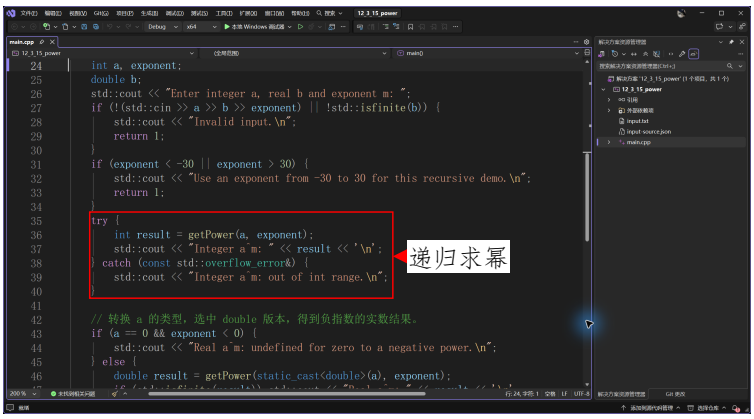
int getPower(int x, int y) {
    if (y < 0) return 0;  // 按题目要求，整数版本负指数返回 0。
    if (y == 0) return 1;
    long long product = static_cast<long long>(getPower(x, y - 1)) * x;
    if (product < std::numeric_limits<int>::min() ||
        product > std::numeric_limits<int>::max()) {
        throw std::overflow_error("int result out of range");
    }
    return static_cast<int>(product);
}

double getPower(double x, int y) {
    if (y == 0) return 1.0;
    if (y < 0) return getPower(x, y + 1) / x;
    return getPower(x, y - 1) * x;
}

int main() {
```

图 67　getPower 的整数与实数重载

整数 getPower 按题目约定将负指数处理为 0，指数为 0 时返回 1；递归得到乘积后检查 int 范围。double 重载分别递归处理正、负指数。



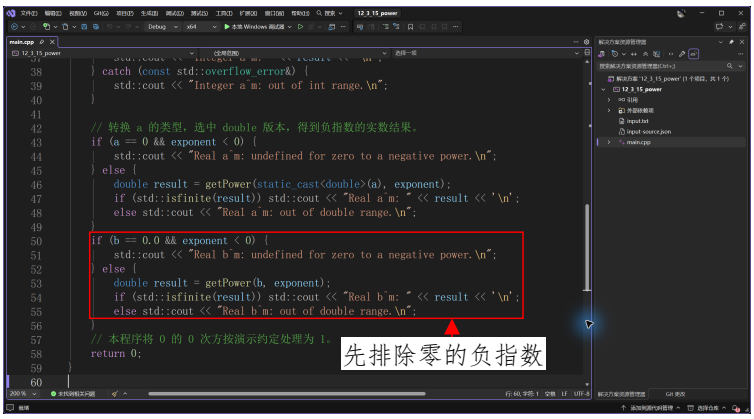

```cpp
    int a, exponent;
    double b;
    std::cout << "Enter integer a, real b and exponent m: ";
    if (!(std::cin >> a >> b >> exponent) || !std::isfinite(b)) {
        std::cout << "Invalid input.\n";
        return 1;
    }
    if (exponent < -30 || exponent > 30) {
        std::cout << "Use an exponent from -30 to 30 for this recursive demo.\n";
        return 1;
    }
    try {
        int result = getPower(a, exponent);
        std::cout << "Integer a^m: " << result << '\n';
    } catch (const std::overflow_error&) {
        std::cout << "Integer a^m: out of int range.\n";
    }

    // 转换 a 的类型，选中 double 版本，得到负指数的实数结果。
    if (a == 0 && exponent < 0) {
        std::cout << "Real a^m: undefined for zero to a negative power.\n";
    } else {
        double result = getPower(static_cast<double>(a), exponent);
```

捕获整数结果越界

图 68　输入检查与整数幂调用

main 先检查输入和指数范围，再调用整数重载。整数结果越界时由异常分支输出提示，不继续将越界值作为结果。

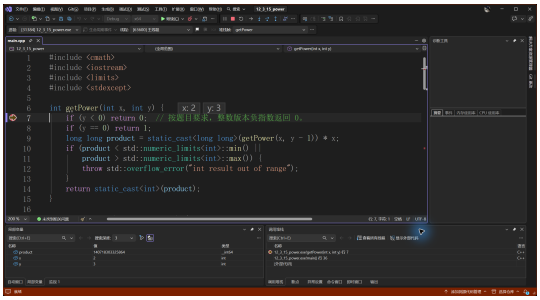



```cpp
    } catch (const std::overflow_error&) {
        std::cout << "Integer a^m: out of int range.\n";
    }

    // 转换 a 的类型，选中 double 版本，得到负指数的实数结果。
    if (a == 0 && exponent < 0) {
        std::cout << "Real a^m: undefined for zero to a negative power.\n";
    } else {
        double result = getPower(static_cast<double>(a), exponent);
        if (std::isfinite(result)) std::cout << "Real a^m: " << result << '\n';
        else std::cout << "Real a^m: out of double range.\n";
    }
    if (b == 0.0 && exponent < 0) {
        std::cout << "Real b^m: undefined for zero to a negative power.\n";
    } else {
        double result = getPower(b, exponent);
        if (std::isfinite(result)) std::cout << "Real b^m: " << result << '\n';
        else std::cout << "Real b^m: out of double range.\n";
    }
    // 本程序将 0 的 0 次方按演示约定处理为 1。
    return 0;
}
```


图 69　实数幂调用与异常处理

通过实参类型选择 double 重载，分别计算 a 和 b 的实数幂。零的负指数单独提示，结果还需检查是否为有限数。

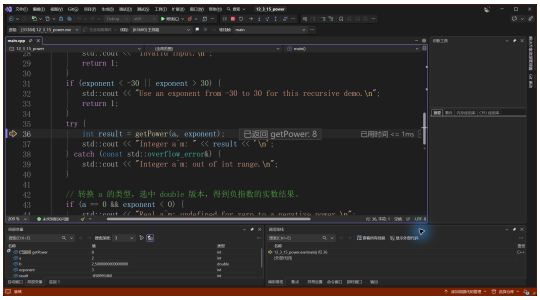


名称	值	类型
product	140718303325864	__int64
x	2	int
y	3	int

名称
12_3_15_power.exe!getPower(int x, int y) 行 7
12_3_15_power.exe!main() 行 36
[外部代码]

图 70　整数重载中的 x=2、y=3

整型重载接收到 x=2、y=3，接下来递归计算；product 尚未初始化，其显示值不作为结果。

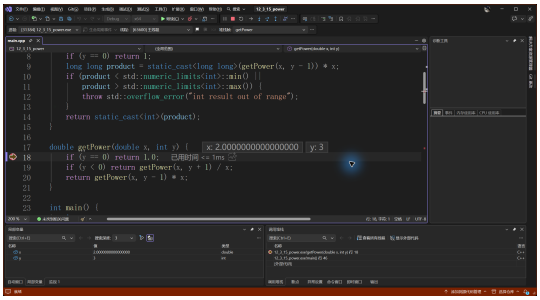


名称	值	类型
已返回 getPower	8	int
a	2	int
b	2.5000000000000000	double
exponent	3	int

名称
12_3_15_power.exe!main() 行 36
[外部代码]

图 71　整数 getPower 的返回值 8

整数重载返回 8，自动窗口中可核对该值。

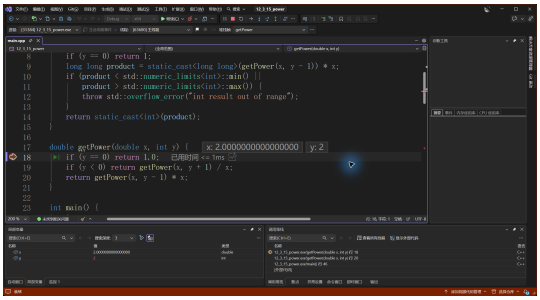


名称	值	类型
x	2.0000000000000000	double
y	3	int

名称
12_3_15_power.exe!getPower(double x, int y) 行 18
12_3_15_power.exe!main() 行 46
[外部代码]

图 72　实数重载形参的值与类型列

static_cast<double>(a)使调用进入实数重载，x=2.0、y=3。

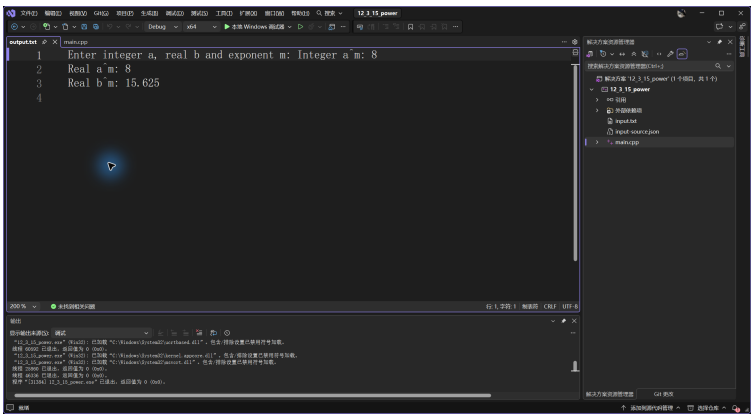


名称	值	类型
x	2.0000000000000000	double
y	2	int

名称
12_3_15_power.exe!getPower(double x, int y) 行 18
12_3_15_power.exe!getPower(double x, int y) 行 20
12_3_15_power.exe!main() 行 46
[外部代码]

图 73　下一层 double 递归中的指数 2

下一层递归中 x 仍为 2.0，y 减为 2；调用堆栈同时显示两层实数重载。




```
1   Enter integer a, real b and exponent m: Integer a^m: 8
2   Real a^m: 8
3   Real b^m: 15.625
```

图 74 VS 运行生成的 power 标准输出文件

输入 2、2.5、3 后，整数 a 的结果为 8，实数 a 的结果为 8，实数 b 的结果为 15.625。

输入 a=2、b=2.5、m=3，整数重载的形参为 2、3，返回值为 8。

static_cast<double>(a)进入 double 重载，形参为 2.0、3；下一层指数变为 2，堆栈可见两层调用。

标准输出中 a 的整数和实数结果均为 8，b 的实数结果为 15.625。

四、实验总结（本次实验的收获、体会、经验和教训）

这次实验把函数的定义、调用和调试放在一起练习。整理程序时，每道可运行题使用独立项目，各自保留一个 main，便于设置启动项目和核对输出。出现编译问题后，先检查函数声明、形参和调用处，再查看错误列表对应的行号，比只盯着最后的生成失败提示更容易找到原因。

阶乘题中，在调用行按 F11 会进入 fact，按 F10 则执行完整调用后回到 main。第一次累乘后 result=5、val=4，返回时 result=120。形参和函数内的局部变量随该次调用结束而离开作用域，main 接收到的是返回值。两个函数都使用 result 这个名字，并不代表它们是同一个变量；把各自的作用域分开，单步时就不会混淆。

引用参数题中，swap(a,a) 判断两个引用的值相等后直接返回；swap(a,b) 先用 temp 保存 7，再完成交换。如果省去暂存步骤，第一次赋值就可能覆盖后面还要用到的旧值。辗转相除同样需要先保存余数，再更新 a、b。对照两段代码，可以看出变量替换前应先明确哪些值还要保留，更新顺序不能随意颠倒。

函数重载要结合实参和形参列表检查。按教师更正，将三个绝对值重载统一改名为 myabs，定义和调用同步修改，再重新生成程序。在三个重载处分别暂停，局部变量窗口和调用堆栈对应 int、double、long，运行结果仍为 4、5.5、6。Add 的两个版本由参数个数区分，不能只改变返回类型就写成另一个重载。

默认参数题的两次调用分别得到 num=100、result=5050，以及 num=10、result=55。省略实参时用的是调用处可见的默认值；参数只能从最右端连续省略。检查错误调用时，fact(3.14) 出现 C4244 警告，但程序仍可生成，实参转成 3，其阶乘为 6；参数缺失或个数错误则不能通过编译。警告和错误需要分别判断，生成成功也不代表类型转换一定符合原来的计算意图。

递归题除了核对最终结果，还要查看调用层次。斐波那契输入 6 时，n=3 的暂停点已有多层调用，n=2 进入基本情形并准备返回 1。求幂时将 a 转为 double 后进入实数重载，下一层指数由 3 变为 2。单步进入后结合调用堆栈查看参数，能把不同层的同名变量分开。先确定终止条件，再跟踪参数怎样靠近它，递归过程就容易理解。

对除零、范围越界、质数边界、负数与零、负指数和整数溢出，也保留了实际检查记录。调试时黄色箭头所在行通常还没有执行，未完成赋值的变量不能当作结果，应执行后再核对。gcd(0,0) 和 0^0 按程序中的演示约定输出；不能表示或没有定义的情况则给出提示。最后将运行生成的 output.txt 与样例结果逐项比较，既检查返回值，也检查输入、分支和输出是否对应。